

Computer Science & Information Technology

C Programming

Weekly Test - 03

Q1 what is the output of the given program?

```
#include<stdio.h>
int f(int);
void main(){
    int i=3,val;
    val= f(i<<4) +f(-i%10) +f(i*2/3+10);
    printf("%d",val);
}
int f(int num){
    return num*5;
}
```

- (A) 240 (B) 285
(C) 93 (D) 120

Q2 Consider the following program

```
#include <stdio.h>
int main()
{
    int i=3;
    switch(i<<5)
    {
        default:i++;
        case 1: i=i+10;
            break;
        case 2:i= i-30;
        case 3: i+-10;
            break;
    }
    printf("%d",i);

    return 0;
}
```

The output of the program is _____

Q3 The value of j at the end of the execution of the following C program is _____

```
int incr (int i)
{
    static int count = 0;
    count = count + i;
    return (count);
}
main () {
    int i,j;
    for (i = 0; i <=4; i++)
        j = incr(i);
}
```

- (A) 10 (B) 4
(C) 6 (D) 7

Q4 Consider the following C function:

```
int f(int n){
    static int i = 1;
    if (n >=5)
        return n;
    n = n+i;
    i++;
    return f(n);
}
```

The value returned by f(1) is

- (A) 5 (B) 6
(C) 7 (D) 8

Q5 What is the output of the following program?

```
#include <stdio.h>
int funcf(int x);
int funcg(int y);
main ()
{
    int x = 5, y = 10, count;
    for (count = 1; count <=2; ++count){
        y+=funcf(x) + funcg(x);
        printf("%d",y);
    }
```



[Android App](#)

| [iOS App](#)

| [PW Website](#)

```

    }
}
funcf (int x) {
    int y;
    y=funcg(x);
    return(y);
}
funcg (int x) {
    static int y = 10;
    y+=1;
    return (y+x);
}

```

- (A) 43 80 (B) 42 74
(C) 33 37 (D) 32 32

Q6 What will be the output of the following C program?

```

#include<stdio.h>
int count1(int n){
    static int x=10;
    return x+n;
}
int count(int n){
    static int d=1;
    d++;
    if(n>1)
        count(n-1);
    d++;
    return count1(d);
}

int main(){
    printf("%d", count(3)+count(4));
    return 0;
}

```

The output of the program is _____

Q7 Consider the following program

```

void Dosomething( int m){
    int n;
    if (m==0)
        return;
    n = m %10;
    if (n!=0)

```

```

    printf("%d", n);
    Dosomething(m/10);
    return ;
}

```

What will be the output of the program when 10234 is given as the input ?

- (A) 43201 (B) 3201
(C) 4321 (D) 1234

Q8 Consider the following program

```

int mystery(int n, int a, int d){
    if (n==1)
        return a;
    else
        return d + mystery(n-1, a, d);
}

```

What will be returned by the called mystery (3, 2, 6)?

- (A) 20 (B) 14
(C) 10 (D) 8

Q9 Consider the following program

```

#include<stdio.h>
int foo (int j){
    static int x = 20;
    x = x + j;
    return x;
}

int main () {
    int i,j=10;

    for(int i =2;i<=4; i<=1 )
        j += foo(foo(foo(10)));

    printf("%d", j);
    return 0;
}

```

The output of the program is _____

Q10 Consider the following program

```

#include<stdio.h>
int foo(int y){
    static int x = 1;
    x++;
    return x+y;
}

```



```

}
int bar(int i){
    return i+10;
}
int main(){
    int x=0,y=0;
    x += foo(bar(10))+y;
    y += bar(foo(11))+x;
    printf("%d\n", (x+y));
    return 0;
}

```

The output of the program is _____

Q11 #include<stdio.h>

```

int foo(int y){
    static int x = 1;
    if (x==y)
        return 10;
    return x+foo(--y);
}
int main(){
    int x=0;
    for (int i=1;i<=5;i++)
        x+=foo(i);
    printf("%d", x);
    return 0;
}

```

The output of the program is

- (A) 60 (B) 70
(C) 80 (D) 90

Q12 Consider the following program

```

#include <stdio.h>
int bar(int a, int r, int n){
    if (n==1)
        return a;
    else
        return a + r*bar(a,r,n-1);
}
int main() {
    printf("%d", bar(1,2,15)-bar(1,2,14) );
    return 0;
}

```

The output of the program is _____

Q13 #include <stdio.h>

```

int bar(int);
int foo(int j){
    static int a = 0;
    if(j==1) return 1;
    a = a + 1;
    return a+bar (j-2);
}
int bar(int i){
    static int a = 0;
    if (i==1) return 1;
    a = a + 1;
    return a+foo(i-1);
}

```

```

int main(){
    printf("%d\n", bar(10));
    return 0;
}

```

What is the output of the above program_____?

- (A) 19 (B) 20
(C) 10 (D) 11

Q14 Consider the following program

```

#include<stdio.h>
int recursionnofun (int p){
    static int count;
    if (p == 0)
        return 0;
    recursionnofun(p-1);
    recursionnofun(p-1);
    recursionnofun(p-1);
    return count+=2 ;
}

```

```

int main(){
    printf("%d ",recursionnofun(3));
    return 0;
}

```

The out put of the program is _____

Q15 Consider the following recursive C function.

```

#include<stdio.h>
void bar(int n) {
    if (n<1) return;
}

```



```
bar (n-1);  
bar(n-2);  
bar (n-3);  
printf("%d", n);  
}
```

```
int main(){  
bar(4);  
return 0;
```

```
}
```

Which of the following is TRUE for the above program

- (A) The output of the program is 12131214
- (B) Number of time bar function is called including call from main is 25
- (C) Number of time bar function is called including call from main is 24
- (D) Output of the program is 12121124

[Android App](#)[iOS App](#)[PW Website](#)

Answer Key

Q1 (B)
Q2 14
Q3 (A)
Q4 (C)
Q5 (A)
Q6 42
Q7 (C)
Q8 (B)

Q9 650
Q10 68
Q11 (A)
Q12 16384
Q13 (A)
Q14 26
Q15 (A, B)



[Android App](#) | [iOS App](#) | [PW Website](#)

Hints & Solutions

Q1 Text Solution:

```
val= f(i<<4) +f(i%10) +f(i*2/3+10);
= 240-15+60 = 285
```

Q2 Text Solution:

Initially $i=3$, which when left-shifted by 5 positions ($i<<5$) becomes 96 in decimal. In the switch(96), no cases match so default case executes first where i becomes 4 ($i++$). Due to no break after default, it falls through to case 1 where i becomes 14 ($i+10$). The break statement in case 1 exits the switch. Finally, printf prints 14 as the output.

Q3 Text Solution:

Static variables retain their values during various function calls.

$i = 0$,	$count =$	0
$i = 1$,	$count =$	1
$i = 2$,	$count =$	$1 + 2 = 3$
$i = 3$,	$count =$	$3 + 3 = 6$
$i = 4$,	$count =$	$6 + 4 = 10$

Q4 Text Solution:

First iteration $f(1)$: $n=2$; $i=2$; $f(2)$ Returns 7

Second iteration $f(2)$: $n=4$; $i=3$; $f(4)$ Returns 7

Third iteration $f(4)$: $n=7$; $i=4$; $f(7)$ Returns 7

P_3 is having no problem with the pointer. P_3 is using dynamic memory allocation to using malloc function. It is the responsibility of the user to release the memory hence no dangling reference will be generated.

Q5 Text Solution:

In the main function the for loop will be executed for $count=1$ and $count=2$.

$count=1$:

```
y+=funcf(x) + funcg(x);
```

$funcf()$ will return 16 and $funcg()$ will return 17.

Final value of y will be 43.

$Count=2$:

```
y+=funcf(x) + funcg(x);
```

$funcf()$ will return 18 and $funcg()$ will return 19.

Final value of y will be 80.

Q6 Text Solution:

We have two static variables: x initialized to 10 in $count1()$ and d initialized to 1 in $count()$. When $count(3)$ is called first, it goes through recursive calls while incrementing d twice in each call. After $count(3)$ completes its execution, d reaches value 7, and $count1(7)$ returns 17 ($10+7$). Next, $count(4)$ executes with d already at 7. It goes through its recursive calls, continuing to increment d twice in each call. After $count(4)$ finishes, d reaches 15, and $count1(15)$ returns 25 ($10+15$). The main function adds the return values of both calls: $count(3) + count(4)$, which is $17 + 25$. Therefore, the program outputs 42.

Q7 Text Solution:

The program uses recursion to process each digit of the input number 10234. At each recursive call, it extracts the last digit using modulo (%) operator and prints it if it's not zero, then makes a recursive call with the remaining digits using integer division (/10).

Starting with 10234, it first prints 4, then processes 1023 printing 3, then 102 printing 2, then moves to 10 (skips printing 0 as per condition), and finally prints 1 from the last digit before terminating when input becomes 0. The digits are printed in reverse order while skipping any zeros, resulting in the output 4321. Therefore, option C (4321) is the correct answer.

Q8 Text Solution:

The function recursively adds d until n becomes 1, at which point it returns a . Evaluating $mystery(3, 2, 6)$, we get $6 + mystery(2, 2, 6)$, which further expands to $6 + (6 + mystery(1, 2,$



Android App

| iOS App

| PW Website

6)). Since $\text{mystery}(1, 2, 6) = 2$, the final result is $6 + 6 + 2 = 14$.

Q9 Text Solution:

The loop runs for $i = 2$ and $i = 4$, meaning two iterations. In the first iteration, $\text{foo}(\text{foo}(\text{foo}(10)))$ executes, where $\text{foo}(10)$ updates x to 30, $\text{foo}(30)$ updates x to 60, and $\text{foo}(60)$ updates x to 120, returning 120. Adding this to j , it becomes $10 + 120 = 130$. In the second iteration, $\text{foo}(\text{foo}(\text{foo}(10)))$ executes again, where $\text{foo}(10)$ updates x to 130, $\text{foo}(130)$ updates x to 260, and $\text{foo}(260)$ updates x to 520, returning 520. Adding this to j , it becomes $130 + 520 = 650$. The final output is 650.

Q10 Text Solution:

the function $\text{foo}(y)$ has a static variable x that starts at 1 and increments with each call. The function $\text{bar}(i)$ simply returns $i + 10$. In $x += \text{foo}(\text{bar}(10)) + y$, $\text{bar}(10)$ returns 20, and $\text{foo}(20)$ updates x to 2 and returns 22, making $x = 22$. In $y += \text{bar}(\text{foo}(11)) + x$, $\text{foo}(11)$ updates x to 3 and returns 14, then $\text{bar}(14)$ returns 24. Adding $x = 22$ to y , we get $y = 46$. Finally, $x + y = 22 + 46 = 68$, so the output is 68.

Q11 Text Solution:

function $\text{foo}(y)$ uses a static variable $x = 1$ and returns 10 when $x == y$, otherwise it recursively calls $\text{foo}(-y)$, adding x to the result. Evaluating $\text{foo}(i)$ for $i = 1$ to 5, we get $\text{foo}(1) = 10$, $\text{foo}(2) = 11$, $\text{foo}(3) = 12$, $\text{foo}(4) = 13$, and $\text{foo}(5) = 14$. The loop in $\text{main}()$ sums these values as $10 + 11 + 12 + 13 + 14 = 60$. Since x is static, it remains 1 throughout, ensuring the same recursive pattern. Hence, the final output is 60, making option A correct.

Q12 Text Solution:

The function $\text{bar}(a, r, n)$ follows a recursive geometric series pattern where $\text{bar}(1, 2, n) = 2^n n - 1$. Calculating for $n = 15$, we get $\text{bar}(1, 2, 15) = 2^{15} \cdot 15 - 1 = 32767$, and for $n = 14$, $\text{bar}(1, 2, 14) = 2^{14} \cdot 14 - 1 = 16383$. The difference $\text{bar}(1, 2, 15) - \text{bar}(1, 2, 14)$

results in $32767 - 16383 = 16384$. This difference follows the pattern of $2^{(n-1)}$, which simplifies to $2^{14} = 16384$. Hence, the final output of the program is 16384.

Q13 Text Solution:

functions $\text{foo}(j)$ and $\text{bar}(i)$ use static variables that retain their values across recursive calls. $\text{bar}(10)$ calls $\text{foo}(9)$, which calls $\text{bar}(7)$, and this alternating pattern continues until reaching the base case. Due to static accumulation, the values build up as $\text{bar}(1) = 1$, leading to $\text{foo}(3) = 4$, $\text{bar}(4) = 7$, $\text{foo}(6) = 9$, $\text{bar}(7) = 11$, $\text{foo}(9) = 12$, and finally $\text{bar}(10) = 19$. The additional increments from static variables contribute to the result, making the final output 19.

Q14 Text Solution:

The function follows a ternary recursion pattern, meaning each call makes three recursive calls until reaching the base case $p == 0$. Since count is static, it retains its value and increments by 2 during each return phase. The number of times count is updated follows a recursive expansion, leading to $\text{count} = 26$ at the final return. This happens due to the growing number of function calls as p decreases from 3 to 0. Therefore, the final output of the program is 26.

Q15 Text Solution:

The function $\text{bar}(n)$ follows a recursive pattern where each call makes three additional recursive calls: $\text{bar}(n-1)$, $\text{bar}(n-2)$, and $\text{bar}(n-3)$. The recursion stops when $n < 1$. To determine the number of calls, we count the number of times $\text{bar}(n)$ is executed, including the initial call from $\text{main}()$. Using recurrence expansion, we get 25 function calls in total. For the output, recursive calls are resolved in depth-first order, and $\text{printf}("%d", n)$; executes only when all recursive calls return. The correct output is 12131214. Thus, option A (output is 12131214) and option B (total calls = 25) are both true.



[Android App](#)

| [iOS App](#)

| [PW Website](#)